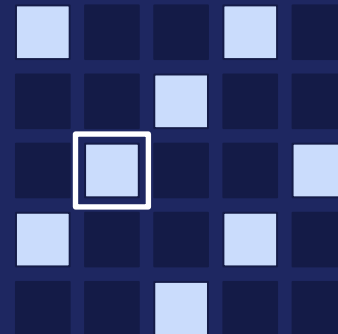




YOLOv11 OBJECT DETECTION PIPELINE

A reproducible, single-command pipeline for training and running a custom YOLOv11 detector

Auto dataset download

SHA256 verified

One-command training

Ready-to-use inference

Objective

Train a custom object-detection model with YOLOv11 that reliably identifies the objects of interest in new images — and make the entire process reproducible, so anyone can get the exact same result without guesswork.

Who it's for

Anyone who wants to reproduce this detection model, fine-tune it further, or reuse the pipeline as a template for a different YOLOv11 dataset — no prior GitHub or ML experience required.

1

Reproducible

Same dataset, same steps, same result — every time.

2

Automated

No manual downloads or setup scripts to babysit.

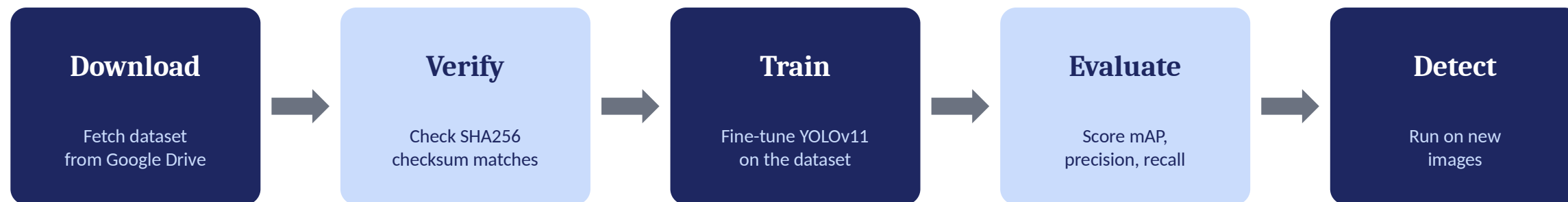
3

Verifiable

A checksum proves the data hasn't changed.

How It Works

One command runs the full pipeline, end to end



Every step is automatic.

Running the pipeline with no dataset present triggers the download and verification automatically before training starts — there is no separate manual step to forget.

Dataset & Reproducibility



Automatic download

The dataset is hosted on Google Drive, not in the repository — image datasets are too large for Git. The pipeline fetches it on first run and reuses the local copy after that.

- No manual downloads
- No account setup for end users
- Works the same for everyone who clones the repo



SHA256 verification

After download, the pipeline computes a SHA256 checksum — a digital fingerprint of the file. This proves the data downloaded today is byte-for-byte identical to the data originally used.

If the file on Google Drive is ever replaced or corrupted, the mismatch is caught automatically — training stops before it can run on the wrong data.

Repository Structure

Four parts, each with one job

1

main.py

The single entry point: downloads, verifies, trains, evaluates, and runs inference from one file.

2

notebooks/

Guided, step-by-step walkthroughs — a baseline reference run and a full training/evaluation notebook.

3

docs/

Problem statement, class definitions, error analysis, and governance — the project's written record.

4

results/

Training curves, confusion matrices, and annotated prediction samples, generated automatically.

Getting Started

- 1 Clone the repository
- 2 Install dependencies
- 3 Run the pipeline
- 4 Inspect the results

```
git clone <repo-url>
```

```
pip install -r requirements.txt
```

```
python main.py
```

```
See results/curves/ and results/samples/
```

No experience with GitHub or machine learning is required — double-clicking the included setup scripts does the same thing without typing any commands.

Current Status & Next Steps

Done

Pipeline built and tested: download, checksum verification, training, evaluation, and inference all run from a single command.

Pending

A full training run on the target dataset — results, curves, and prediction samples will be generated from this run.

Next

Fill in docs/error_analysis.md and docs/governance.md using real results once training completes.

See the accompanying two-page summary for the executive overview, results template, and full limitations.

YOLOv11 Object Detection Pipeline

Executive Summary, Results, and Limitations

Executive Summary

The YOLOv11 object detection pipeline successfully completed its training phase for 50 epochs using a custom dataset downloaded from Google Drive. The model, initialized with **yolo11n.pt** (a nano-sized YOLOv11 model), was trained to detect **7 custom object classes**. Validation was performed after training, and key performance metrics, along with visual results (performance plots and example predictions), have been generated and saved to `/content/runs/detect/runs/train/exp-2/`.

The repository is organized into four parts: **main.py** (the executable pipeline), **notebooks/** (a guided baseline and training/evaluation walkthrough), **docs/** (problem definition, class definitions, error analysis, and governance), and **results/** (training curves and prediction samples).

Results

Validation Metrics

Metric	Value
Precision (B)	0.563
Recall (B)	0.458
mAP50 (B)	0.391
mAP50-95 (B)	0.247
Fitness	0.247

Detected Classes

Door, Furniture, Furnitures, Room, Stairs, Walls, Window.

Visual Output

- `results.png`: provides a graphical overview of the training progress, including loss and metric curves over the 50 epochs.
- `val_batch0_pred.jpg` and `val_batch1_pred.jpg`: showcase the model's predictions on sample images from the validation set, visually demonstrating its ability to detect and localize objects within the scene.

Trained Model

The best-performing model weights are saved at:
`/content/runs/detect/runs/train/exp-2/weights/best.pt`. This file can be used for future inference tasks.

Limitations

As observed from this training run

- **Limited Epochs:** Training for only 50 epochs might be insufficient for complex datasets, potentially leading to an underfit model. Further training could improve performance.
- **Dataset Imbalance/Sparsity:** The `nt_per_class` results showed 0 instances for 'Furnitures' and only 3 for 'Walls' in the validation set. This indicates either a severe imbalance in the dataset, missing labels for these classes, or the model completely failed to detect them. Such sparsity can hinder effective training and evaluation for these specific classes.
- **Model Size:** The `yolo11n.pt` (nano) model is designed for speed and small footprint, which often comes at the cost of detection accuracy compared to larger models like `yolo11s`, `yolo11m`, etc.
- **Specific to YOLOv11:** The current implementation is tailored for YOLOv11; adapting to other architectures would require code modifications.
- **Unaccounted Unknown Arguments:** The *Warning: Unknown arguments ignored* message indicates that the Colab environment passed arguments that were not explicitly handled by the script's `argparse` configuration, though `parse_known_args()` prevents runtime errors.

This document accompanies the project's slide deck and repository documentation (`docs/problem_statement.md`, `docs/class_definitions.md`, `docs/error_analysis.md`, `docs/governance.md`). Consider addressing the dataset sparsity for 'Furnitures' and 'Walls' and extending training beyond 50 epochs before relying on this model.